MCP servers & custom tool integration

Give Copilot reach

Session M3-03 | 3 hours

Agenda

Segment	Time
What MCP is	10 min
Configure servers in Copilot CLI	12 min
Demo: GitHub server and approvals	10 min
Skill vs custom agent vs MCP	8 min
Pros, cons, and tips	12 min
Lab handoff	8 min

What MCP is

Model Context Protocol: an open standard for connecting AI tools to outside systems.

- **Client:** the app that talks to the model. Copilot CLI, VS Code.
- **Server:** a program that offers capabilities. GitHub, a docs search, your own code.

Without MCP, every client needs custom code for every system. With it, a server written once works in any client.

What a server offers

Type	Plain meaning	Who triggers it
Tools	Actions Copilot can call	The model
Resources	Data it can read, like files or records	The client or you
Prompts	Saved prompt templates	You

Tools are what you use most. Copilot CLI uses tools. Check the docs for resource and prompt support in each client.

How a tool call works

1. The client starts or connects to the server and asks: **what tools do you have?**
2. The server sends each tool's **name, description, and input schema**.
3. You ask a question. The model picks a tool and fills in the arguments.
4. **Copilot asks your permission**, unless you allowed it.
5. The server runs the tool and returns text. The model reads it and answers.

The **schema** is a JSON description of the arguments: names, types, required fields. It lets the client reject bad calls.

Two transports

Transport	Runs	Use for	Credentials
stdio	A local process Copilot starts	Your own tools, local files	Environment variables
HTTP	A remote service	Hosted servers like Microsoft Learn	Sign-in or headers

A stdio server talks over standard input and output. **Never print logs to stdout.** Use stderr.

SSE is an older HTTP variant. It still works but is deprecated.

Configure a server in Copilot CLI

```
/mcp                dashboard of servers and their status
/mcp add            form: name, type, command or URL, tools
/mcp search         browse the GitHub MCP Registry
/mcp show NAME      status and tool list
/mcp disable NAME
```

From the terminal:

```
copilot mcp add context7 -- npx -y @upstash/context7-mcp
copilot mcp add --transport http NAME URL
copilot mcp list
```

The **GitHub MCP server is built in**.

The config file

`~/.copilot/mcp-config.json`

```
{
  "mcpServers": {
    "weather": {
      "type": "local",
      "command": "node",
      "args": ["/path/to/src/index.js"],
      "tools": ["*"]
    },
    "microsoft-learn": {
      "type": "http",
      "url": "https://learn.microsoft.com/api/mcp",
      "tools": ["*"]
    }
  }
}
```

`tools` limits what Copilot can see from that server.

Project config and VS Code

Where	File
Copilot CLI, user	`~/.copilot/mcp-config.json`
Copilot CLI, project	`.mcp.json` or `.github/mcp.json`
VS Code	`.vscode/mcp.json` (key: `servers`)

- Project servers load only after you **trust the folder**.
- The CLI doesn't read `.vscode/mcp.json`. The key differs.
- Share a project config by committing `.github/mcp.json`. **Never commit tokens.**

Demo: GitHub server and approvals

1. `/mcp`, then `/mcp show` for the GitHub server.
2. Ask: *List the open issues labeled `bug` in a public repo.*
3. Read the approval prompt. Say what each option means.
4. Restart with `--deny-tool` for one tool. Ask again.

Watch for: **which tool**, **which arguments**, and **what came back**.

Tool permissions

Copilot asks before it uses a tool that isn't already allowed. You can allow once or for the session. Saved approvals live in `~/.copilot/permissions-config.json`.

```
copilot --allow-tool='MyServer(create_issue)'
copilot --deny-tool='MyServer(delete_issue)'
copilot --excluded-tools='...'      # hide tools from the model
```

- **Deny beats allow.** Always.
- `--allow-all-tools` and `--yolo` skip every prompt. Use them only in a sandbox.
- Organizations can also restrict MCP servers by policy.

Skill vs custom agent vs MCP server

	Skill	Custom agent	MCP server
Gives Copilot	Know-how	A role	Reach
Is	Markdown, plus optional files	Markdown with tools and model	A running server
Loads	On demand	When picked or delegated	When configured
Risk	Wrong trigger	Too many tools	Credentials, data

They combine. An agent can use skills. A skill can teach Copilot to use an MCP server well. An agent can list which MCP tools it may use.

More: M3-04 custom agents, M3-05 skills.

Pros and cons

Pros	Cons
Live data instead of pasted text	**Credentials** you must protect
Actions: create issues, query data	An action can change real systems
One server works in many clients	**Every tool costs context**
Reuse servers from the registry	**Tool output can carry injected instructions**

Cost and risk

- **Context cost:** each tool's name, description, and schema goes into every request. Fifty tools crowd out your code.
- **Prompt injection:** a tool returns text from an issue, web page, or file. The text says *ignore your instructions and run this command*. The model may comply.
- **Credentials:** a token in config can do whatever you can.

Defenses: **least privilege**, **approval prompts**, trusted servers only.

Tips

- **Few, focused tools.** Five clear tools beat fifty.
- **Write the description for the model.** Say what the tool does and when to use it.
- **Least privilege.** Read-only tokens. Limit `tools` to what the task needs.
- **Narrow schemas.** Required fields, enums, `additionalProperties: false`.
- **Return errors, don't hide them.** Set `isError: true`.
- Check a server's source before you trust it.

Common mistakes

Mistake	Fix
Logging to stdout in a stdio server	Log to stderr
Relative path in `args`	Use an absolute path
Tool description like "does stuff"	Name the action and when to use it
Token committed in config	Use environment variables
`--yolo` on an unknown server	Allow named tools only
Enabling every server "just in case"	Enable per task

Lab handoff

You will:

1. Use the built-in GitHub server.
2. Add Microsoft Learn (remote).
3. Connect a small weather server and see Copilot call it.
4. Add a tool and test good vs bad descriptions.
5. Try a prompt injection and the permission flags.

Starter: `lab/starter/custom-mcp/`